

Mitigare Rowhammer

Progetto di ICT Risk Assessment

Francesca Pinna e Sofia Pisani

Università di Pisa

28 giugno 2026

① Rowhammer

② PARA

③ Graphene

④ Blockhammer

⑤ Conclusioni

Cosa è Rowhammer

- Rowhammer è l'attacco che sfrutta la vulnerabilità delle DRAM commerciali agli errori da interferenza.
- Permette ad un attaccante di modificare il contenuto di alcune celle di memoria senza averne accesso *nè in lettura, nè in scrittura*, ma attraverso accessi ripetuti alle righe di memoria adiacenti.
- L'esistenza degli errori da interferenza è nota da tempo ai produttori di DRAM, che tentano di limitarne l'impatto, ma il primo studio ad analizzare le sue conseguenze sulla sicurezza è stato *Flipping Bits in Memory Without Accessing Them: An Experimental Study of DRAM Disturbance Errors* nel 2014. [1]

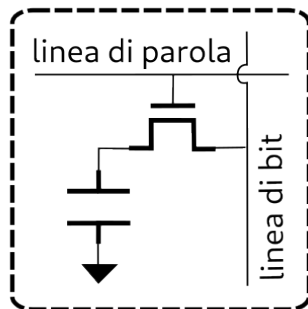
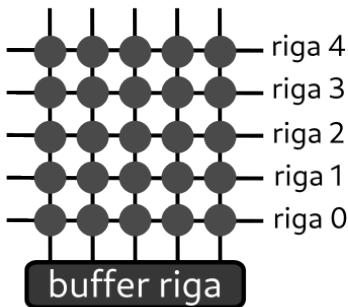
Perché è un problema attuale

I progressi nella tecnologia DRAM permettono di piazzare celle di memoria più piccole sempre più vicine l'una all'altra. Questo riduce il costo per bit, ma la crescita della densità ha un impatto negativo sull'affidabilità della memoria.

- Una cella più piccola può accumulare una quantità limitata di carica, con una riduzione del margine di rumore e maggiore suscettibilità alla perdita di dati;
- Aumenta il numero di celle anomale che sono suscettibili all'interferenza elettromagnetica.

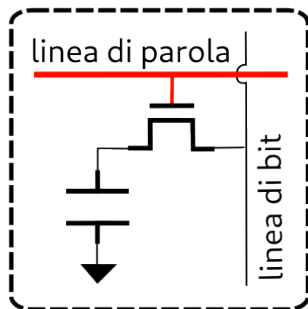
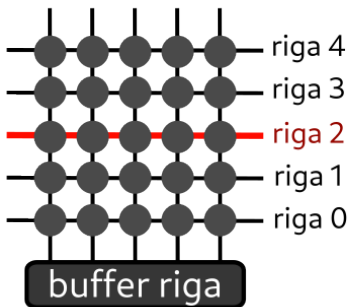
Quindi la DRAM ad alta densità ha più probabilità di essere soggetta ad **interferenza**, un fenomeno in cui le celle influenzano l'una l'operazione dell'altra.

DRAM



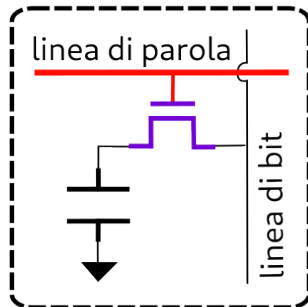
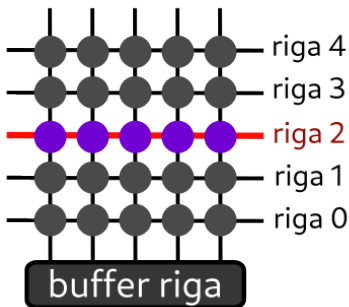
La memoria DRAM è formata da matrici bidimensionali organizzate in righe e colonne. Ogni cella è formata da un condensatore e da almeno un transistor di accesso. A seconda di se il condensatore è carico o scarico, la cella tiene un valore 1 o 0.

DRAM



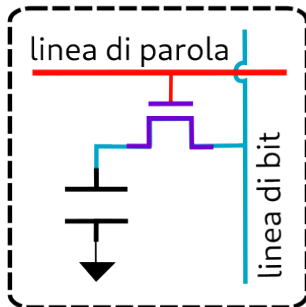
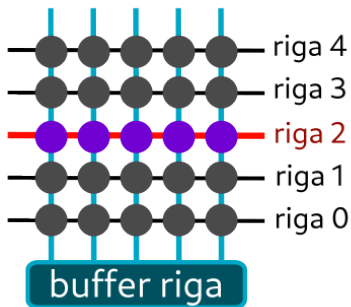
Ogni cella sta nell'intersezione di due linee perpendicolari: la linea di parola orizzontale e la linea di bit verticale. La linea di parola connette tutte le celle nella direzione orizzontale, la riga, e la linea di bit tutte le celle della direzione verticale, la colonna.

DRAM

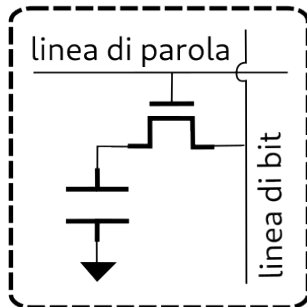


Quando la linea di parola è portata a tensione alta, attiva tutti i transistori di accesso nella riga, che connette tutti i condensatori alle loro linee di bit.

DRAM



Questo permette ai dati di tutta la riga di essere trasferiti al buffer di riga sotto forma di carica. Il buffer di riga legge le cariche delle celle (processo che distrugge i dati in esse contenuti) e immediatamente riscrive gli stessi valori.



Tutti gli accessi alla riga sono svolti dal buffer di riga invece che direttamente dalla memoria. Quando non ci sono più accessi alla riga, la linea di parola è portata a tensione bassa, disconnettendo i condensatori dalle linee di bit.

Memory controller

Accesso alla DRAM

- ➊ **Apertura della riga:** la riga è aperta quando si attiva la sua linea di parola, trasferendo tutti i suoi dati nel buffer.
- ➋ **Lettura o scrittura delle colonne:** si accede ai dati contenuti nel buffer di riga, leggendo o scrivendo in tutte le colonne necessarie.
- ➌ **Chiusura:** Prima che si possa aprire una riga diversa dello stesso gruppo, la riga originale deve essere chiusa, abbassando la sua linea di parola e ripulendo il buffer.

Il memory controller gestisce questi passaggi con tramite comandi come *activate*, *read/write*, *precharge*(chiusura). Ha anche il ruolo di ordinare la ricarica delle celle con il comando *refresh*, necessaria perché la carica accumulata in una cella DRAM non è permanente.

Errori da interferenza

- Quando ci sono numerosi accessi non consecutivi alla stessa riga di memoria, la linea di parola si deve attivare e disattivare ripetutamente.
- Queste fluttuazioni di tensione hanno un effetto di interferenza sulle righe adiacenti, facendogli perdere carica ad una velocità maggiore.
- Se una cella perde troppa carica prima di essere ripristinata al suo valore originale, subisce un **errore da interferenza** che si manifesta in un **Bit flip**.
- Le righe direttamente adiacenti hanno l'effetto maggiore, ma anche le righe a distanze superiori possono causare errori da interferenza.
- Gli errori si verificano solo dopo un certo numero di attivazioni, che chiamiamo **soglia RowHammer**.

Errori da interferenza (cont.)

Gli errori di interferenza sono particolarmente gravi perchè violano due invarianti essenziali di una memoria:

- Un accesso in lettura non deve modificare i dati;
- Un accesso in scrittura deve modificare solo i dati all'indirizzo in cui scrive.

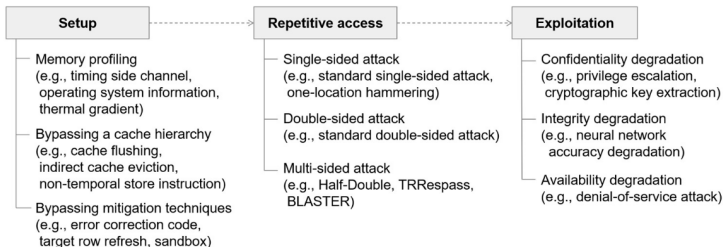


Figura: Passi di un'intrusione

Caratteristiche dell'attacco

- Se una riga di memoria è aperta ripetutamente, sia gli accessi in lettura che in scrittura possono indurre errori RowHammer, che avvengono tutti in righe diverse da quelle a cui si fa l'accesso.
- Siccome diverse righe DRAM sono mappate a pagine software diverse, un programma a livello utente può corrompere bit specifici in pagine che appartengono ad altri programmi.
- Un programma malevolo può usare i bit flip per violare le protezioni di memoria e compromettere il sistema, iniettando errori in altri programmi, crashare il sistema o prenderne il controllo.

Caratteristiche dell'Attacco II

Precondizioni

- **Permessi:** Permessi di scrittura o lettura su una cella di memoria fisicamente adiacente al bersaglio.
- **Informazioni:** Disposizione delle celle vulnerabili e mappa parziale da indirizzi virtuali a fisici.

Postcondizione: Permessi di scrittura sulla riga bersaglio con limitazione ad un bit-flip.

Gli attacchi Rowhammer sono remoti, automatici e basati su una vulnerabilità al livello hardware, che non può essere patchata se non dal produttore.

Esempi di attacchi

- Nel 2015, Google Project Zero ha dimostrato che la vulnerabilità può essere utilizzata da programmi a livello utente per ottenere privilegi kernel in esempi reali. [2]
- Lo studio Drammer(2016) ha provato che anche i dispositivi mobili Android sono vulnerabili agli attacchi. [3]
- Gruss et al. (2016) ha mostrato che è possibile prendere controllo come root di un server remoto tramite codice javascript, che potrebbe essere eseguito sul browser. [4]
- Numerosi altri studi hanno sfruttato RowHammer tramite il protocollo Remote Direct Memory Access (RDMA).
- Uno studio ha dimostrato che 13 bit-flips sui pesi dei parametri in DRAM erano sufficienti a degradare l'accuratezza di una rete neurale convoluzionale dal 68.9% allo 0.1% [5]

Rowhammer su reti neurali

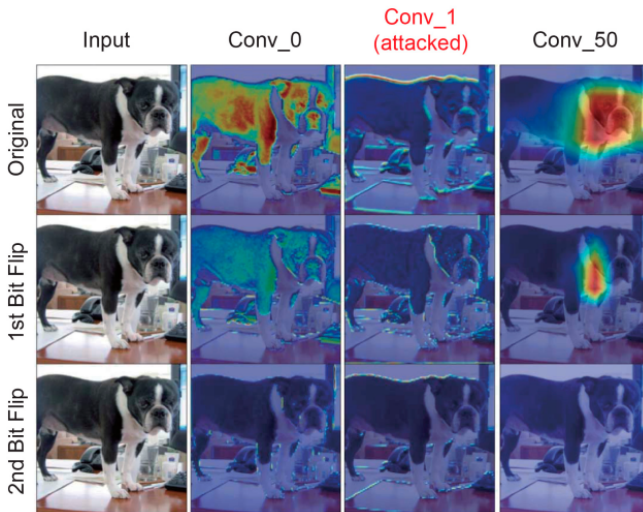


Figura: Degradazione riconoscimento immagini [6]

Soluzioni Immediate

- Fare o comprare memorie migliori.
- ECC non è sufficiente per correggere gli errori causati da un attacco RowHammer. [7]
- Aumentare la frequenza di ricariche: maggior consumo di energia, peggiore performance.
- Identificare le celle suscettibili e rimappare. Richiede molto tempo e rischia di togliere buona parte della memoria.
- L'utente potrebbe trovare e aumentare la frequenza di ricarica delle celle vittima, ma sarebbe inefficiente.
- Contare le righe attivate frequentemente: utilizzabile se si trova una soluzione efficiente in spazio. [1]

Probabilistic Adjacent Row Activation (2014)

Attivazione probabilistica di riga adiacente

- È una soluzione a livello hardware che agisce nel memory controller.
- Ogni volta che una riga è aperta e chiusa, una delle sue righe adiacenti è anch'essa aperta e chiusa (cioè ricaricata) con una probabilità bassa.
- La probabilità è calcolata in base alla soglia RowHammer. Quindi se una riga in particolare è chiusa e aperta molte volte, diventa statisticamente certo che anche le righe adiacenti saranno ricaricate, così da prevenire i bit-flip.

Vantaggi e svantaggi

- Il vantaggio principale di PARA è che è *senza stato*. Non richiede strutture dati in hardware che contino il numero di volte che le righe sono state aperte o per tenere gli indirizzi delle righe aggressore/vittima.
- PARA è un algoritmo probabilistico: non può dare garanzie di correttezza ma può avere precisione arbitraria modificando la probabilità di attivazione.
- Ha un overhead costante perché non bersaglia le attivazioni ripetute, e non distingue scenari normali dai tentativi di attacco.
- Più la soglia RowHammer si abbassa, maggiore è l'overhead di PARA.

Prestazioni di PARA

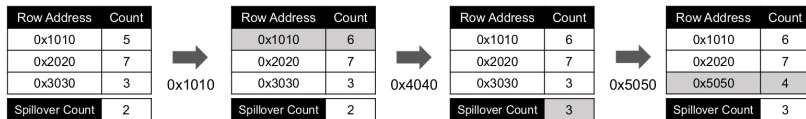
Nello studio originale del 2014, PARA fu testato su un simulatore di DRAM DDR3, su 29 workloads considerati memory-intensive.

- Da subito lo applicarono su dieci possibili righe adiacenti, invece di solo due, in modo da considerare i re-mappings.
- La probabilità di refresh scelta fu $p = 0.005$.
- In media, ci fu una degradazione dello 0.197% sul throughput di operazioni, con un picco di 0.745%.

Non abbiamo trovato molti esperimenti con PARA su memorie più recenti, ma sappiamo che la sua performance degrada alla riduzione della soglia RowHammer, quindi possiamo prevedere che scalerà male con l'aumento della densità delle memorie.

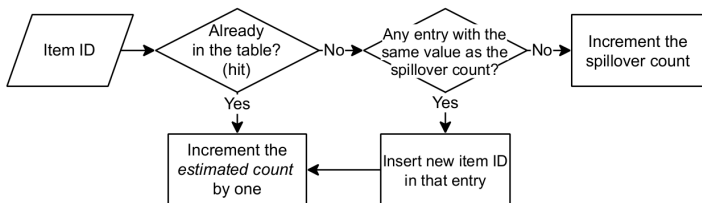
Graphene (2020)

- Graphene è una tecnica di mitigazione di Rowhammer che identifica le righe attivate più frequentemente e le ricarica in modo selettivo. [8]
- L'algoritmo di Misra-Gries è un algoritmo classico di streaming che garantisce di identificare gli elementi più frequenti in un flusso di dati continuo.
- Graphene trova tutte le righe attivate più di una certa soglia e le ricarica.



L'algoritmo di Misra-Gries

L'algoritmo di Misra-Gries usa una tabella di dimensione N e un contatore di spillover per tenere traccia degli elementi più frequenti in uno stream. La tabella contiene tutti gli indirizzi di memoria che sono stati aperti più di $\frac{W}{N+1}$ volte nel ciclo di refresh corrente, dove W è il numero di aperture di celle di memoria possibili in un ciclo di refresh.

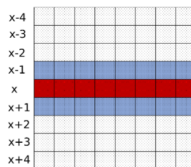
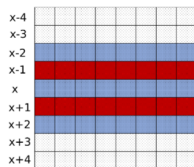


Vantaggi e svantaggi

- Protezione garantita - Graphene ricarica le righe vittima prima di raggiungere la soglia RowHammer.
- Piccolo overhead di energia e performance - Graphene non genera ricariche addizionali in task realistiche, non malevole. Anche in caso di attacco, il numero di ricariche è piccolo.
- Necessita di una zona di memoria relativamente ampia.
- Scalabilità - scala bene per diverse soglie di RowHammer, adattandosi alle evoluzioni della tecnologia.

Problema degli aggressori multipli

- Gli attacchi RowHammer possono sfruttare l'effetto di interferenza tra righe non direttamente adiacenti, cioè a distanza maggiore di uno.
- Le prime strategie di mitigazione non erano capaci di gestire gli attacchi multi-sided, che possono eludere anche alcune strategie di protezione moderne.
- Sia PARA che Graphene possono essere adattate per proteggere righe non direttamente adiacenti, ma è necessario che il produttore della memoria riveli quali righe sono fisicamente vicine a quali.



Blockhammer(2021)

RowBlocker

Blockhammer[9] consiste di due componenti, RowBlocker e AttackThrottler.

- RowBlocker previene la possibilità di bit-flip, perché rende impossibile raggiungere la soglia RowHammer in un singolo ciclo di ricarica.
- Usa un Counting Bloom Filter doppio per tenere traccia della frequenze di attivazione delle righe. Quando una di queste supera una soglia prestabilita, il RowBlocker la segna come *blacklisted* e limita le sue attivazioni per un intervallo di tempo detto *epoch*.
- Sono limitate le attivazioni ripetute solo delle righe considerate pericolose.

AttackThrottler

- AttackThrottler allevia la degradazione di performance che un attacco può causare alle applicazioni benigne, riducendo l'utilizzo di memoria dei thread malevoli.
- Monitora la metrica *RowHammer Likelihood Index* (RHLI) per ogni thread ed ogni bank, che quantifica la somiglianza tra il pattern di accesso del thread e quello di un attacco RowHammer reale.
- RHLI è il rapporto tra il numero di attivazioni di righe *blacklisted* e il numero massimo di volte che una riga blacklisted può essere attivata con BlockHammer attivo.
- L'RHLI di un thread normale è pari a zero.
- AttackThrottler limita il numero di richieste di accesso alla memoria in modo proporzionale al RHLI del thread.

Vantaggi e svantaggi

- Blockhammer agisce completamente all'interno del memory controller e non richiede informazioni proprietarie o modifiche al chip DRAM, a differenza di soluzioni commerciali come TTR.
- Non ha neanche bisogno di conoscere l'adiacenza fisica delle righe in memoria, perché agisce sulla riga aggressore, invece che sulla vittima.
- Blockhammer è progettato per superare gli ostacoli di overhead e memoria delle protezioni da RowHammer.
- Blockhammer non rallenta tutti gli accessi ripetuti ad una riga, ma solo quelli che superano una certa soglia, proteggendo le applicazioni benevole.
- Si evita dunque un possibile utilizzo di RowHammer per un attacco Denial of Service.

Soluzioni software

Le strategie di mitigazione si possono implementare al livello software, al livello del memory controller, o direttamente all'interno del chip DRAM.

- Il livello software non conosce la disposizione fisica delle righe, quindi deve fare supposizioni pericolose. [10]
- Noi abbiamo presentato strategie che agiscono al livello del memory controller, perché hanno maggiori informazioni sul funzionamento interno della memoria.
- Le soluzioni hardware hanno limiti alla complessità degli algoritmi da implementare, e non sono facilmente studiabili in quanto proprietarie.

Se il difensore non sa quali sono le righe adiacenti, le soluzioni che ricaricano selettivamente le righe vittima non possono essere completamente affidabili.

Soluzioni Hardware

DRAM o Memory Controller?

- Sia le soluzioni implementate nel memory controller che quelle implementate nella DRAM non possono essere utilizzate in sistemi legacy, perché è necessario cambiare le componenti fisiche.
- Lo scopo di un memory controller è di gestire il traffico verso la memoria, mentre quello del chip DRAM è di far stare più bit possibili in ogni millimetro quadro.
- Tutto lo spazio aggiuntivo dedicato alla logica è considerato spazio perso per il produttore di DRAM.

Soluzioni commerciali: TRR

2016 in poi

- I chip di DRAM della generazione corrente, dalla DDR4, sostengono di essere "RowHammer free".
- Implementano direttamente sul chip una soluzione chiamata TRR (Target Row Refresh), che effettua ricariche strategiche su possibili righe vittime.
- Non essendo pubblica la sua implementazione in DRAM, è difficile comprendere funzionamento ed effettività della soluzione, ma sappiamo che la sua implementazione cambia da venditore a venditore, da chip a chip.
- Studi di reverse engineering hanno tratto delle osservazioni sul suo funzionamento, ed individuato delle mancanze nella sua protezione dagli attacchi RowHammer multi-sided. [11]

Simulazione

La simulazione ha scopo di dare una rappresentazione grafica di RowHammer e delle sue tecniche di mitigazione, ma non può dare dati numerici accurati all'esempio reale.

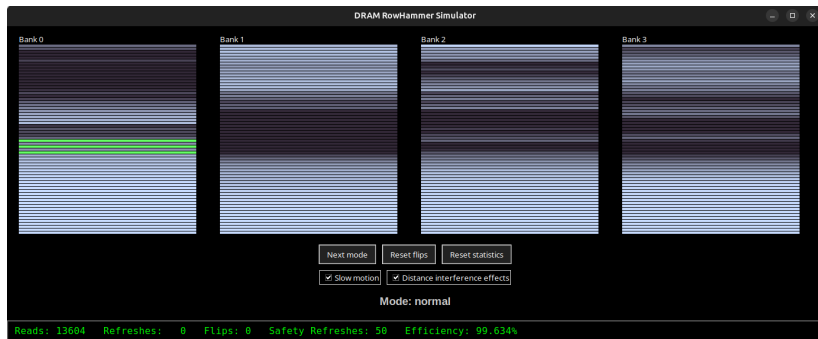


Tabella di paragone

Strategia	Overhead memoria	Overhead prest.	Layout fisico	Protezione multi-sided
PARA	–	1.2%	✓	≈
Graphene	5.22 kB	0%	✓	≈
BlockHammer	53.21 kB	0.5%	×	✓

(1) Overhead prestazionale in assenza di attacchi. (2) ≈ protezione solo approssimativa contro attacchi multi-sided.

Parametro	Valore
Soglia RowHammer	32k
Finestra di refresh (T_{refw})	64 ms
Delay tra attivazioni (T_{rc})	46.25 ns
Numero massimo di attivazioni in un ciclo di refresh	1.3×10^6
Numero di banchi	16

Tabella: Caratteristiche della memoria DDR4.

Conclusioni

- Tutte le strategie trattate hanno la limitazione che, per garantire la correttezza, devono conoscere la soglia RowHammer della memoria su cui agiscono, il che non è sempre possibile.
- Abbiamo presentato la versione più elementare di PARA, che è superata in prestazioni da Graphene e Blockhammer. [9]
- Tutte e tre le strategie possono essere utilizzate per mitigare l'effetto della vulnerabilità.
- Rowhammer è un problema difficile perché ci sono forti limitazioni dall'hardware, ad esempio sul numero di refresh.
- Perché si trovino soluzioni complete al problema, è necessaria la collaborazione con i produttori di hardware.

Bibliografia I

- [1] Yoongu Kim et al. «Flipping bits in memory without accessing them: An experimental study of DRAM disturbance errors». In: *2014 ACM/IEEE 41st International Symposium on Computer Architecture (ISCA)*. 2014, pp. 361–372. DOI: 10.1109/ISCA.2014.6853210.
- [2] Mark Seaborn e Thomas Dullien. *Exploiting the DRAM Rowhammer Bug to Gain Kernel Privileges*. <https://projectzero.google/2015/03/exploiting-dram-rowhammer-bug-to-gain.html>. Google Project Zero Blog. Mar. 2015.

Bibliografia II

- [3] Victor van der Veen et al. «Drammer: Deterministic Rowhammer Attacks on Mobile Platforms». In: *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*. DIMVA. Vienna, Austria, ott. 2016.
- [4] Daniel Gruss, Clémentine Maurice e Stefan Mangard. «Rowhammer.js: A Remote Software-Induced Fault Attack in JavaScript». In: *CoRR* abs/1507.06955 (2015). arXiv: 1507.06955. URL: <http://arxiv.org/abs/1507.06955>.
- [5] Adnan Siraj Rakin, Zhezhi He e Deliang Fan. «Bit-Flip Attack: Crushing Neural Network with Progressive Bit Search». In: *CoRR* abs/1903.12269 (2019). arXiv: 1903.12269. URL: <http://arxiv.org/abs/1903.12269>.

Bibliografia III

- [6] Dahoon Park et al. *ZeBRA: Precisely Destroying Neural Networks with Zero-Data Based Repeated Bit Flip Attack*. 2021. arXiv: 2111.01080 [cs.LG]. URL: <https://arxiv.org/abs/2111.01080>.
- [7] Lucian Cojocar et al. «Exploiting Correcting Codes: On the Effectiveness of ECC Memory Against Rowhammer Attacks». In: *2019 IEEE Symposium on Security and Privacy (SP)*. 2019, pp. 55–71. DOI: 10.1109/SP.2019.00089.
- [8] Yeonhong Park et al. «Graphene: Strong yet Lightweight Row Hammer Protection». In: *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 2020, pp. 1–13. DOI: 10.1109/MICRO50266.2020.00014.

Bibliografia IV

- [9] A. Giray Yağlikçi et al. «BlockHammer: Preventing RowHammer at Low Cost by Blacklisting Rapidly-Accessed DRAM Rows». In: *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 2021, pp. 345–358. DOI: 10.1109/HPCA51647.2021.00037.
- [10] Yueqiang Cheng et al. «CATTmew: Defeating Software-Only Physical Kernel Isolation». In: *IEEE Transactions on Dependable and Secure Computing* 18.4 (2021), pp. 1989–2004. ISSN: 2160-9209. DOI: 10.1109/tdsc.2019.2946816. URL: <http://dx.doi.org/10.1109/TDSC.2019.2946816>.
- [11] Pietro Frigo et al. *TRRespass: Exploiting the Many Sides of Target Row Refresh*. 2020. arXiv: 2004.01807 [cs.CR]. URL: <https://arxiv.org/abs/2004.01807>.